

Playfair Cipher

Aim

To implement the **Playfair Cipher encryption technique** using a keyword-based 5×5 matrix in C for secure encryption of plaintext.

Algorithm

1. **Start** the program.
2. Enter the keyword and plaintext.
3. Convert all letters of the keyword and plaintext to uppercase.
4. Remove duplicate letters from the keyword.
5. Construct a **5 × 5 Playfair matrix** using the keyword followed by the remaining alphabets.
 - Combine **I** and **J** into one letter.
6. Divide the plaintext into pairs (digraphs).
7. If both letters in a pair are the same, insert **X** between them.
8. If the plaintext has an odd number of letters, append **X** at the end.
9. Find the positions of both letters in the Playfair matrix.
10. Encrypt each pair using the following rules:
 - **Same Row:** Replace each letter with the letter to its right (wrap around if needed).
 - **Same Column:** Replace each letter with the letter below it (wrap around if needed).
 - **Rectangle Rule:** Replace each letter with the letter in the same row but in the other letter's column.
11. Display the encrypted ciphertext.
12. **Stop** the program.

Program :

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
char matrix[5][5] = {  
    {'M','O','N','A','R'},  
    {'C','H','Y','B','D'},  
    {'E','F','G','T','K'},
```

```
    {'L','P','Q','S','T'},  
    {'U','V','W','X','Z'}  
};
```

```
void findPosition(char ch, int *row, int *col)
```

```
{  
    if(ch == 'J')  
        ch = 'I';  
  
    for(int i = 0; i < 5; i++)  
    {  
        for(int j = 0; j < 5; j++)  
        {  
            if(matrix[i][j] == ch)  
            {  
                *row = i;  
                *col = j;  
                return;  
            }  
        }  
    }  
}
```

```
int main()
```

```
{  
    char plaintext[100];  
    char prepared[100];  
    char ciphertext[100];  
    int i, j = 0;  
  
    printf("Enter Plaintext: ");  
    scanf("%s", plaintext);
```

```

// Convert to uppercase
for(i = 0; plaintext[i] != '\0'; i++)
    plaintext[i] = toupper(plaintext[i]);

// Prepare plaintext
for(i = 0; plaintext[i] != '\0'; i++)
{
    prepared[j++] = (plaintext[i] == 'J') ? 'T' : plaintext[i];

    if(plaintext[i + 1] != '\0' &&
        prepared[j - 1] == ((plaintext[i + 1] == 'J') ? 'T' : toupper(plaintext[i + 1])))
    {
        prepared[j++] = 'X';
    }
}

if(j % 2 != 0)
    prepared[j++] = 'X';

prepared[j] = '\0';

// Encryption
for(i = 0; i < j; i += 2)
{
    int r1, c1, r2, c2;

    findPosition(prepared[i], &r1, &c1);
    findPosition(prepared[i + 1], &r2, &c2);

    if(r1 == r2)
    {

```

```

        ciphertext[i] = matrix[r1][(c1 + 1) % 5];
        ciphertext[i + 1] = matrix[r2][(c2 + 1) % 5];
    }
    else if(c1 == c2)
    {
        ciphertext[i] = matrix[(r1 + 1) % 5][c1];
        ciphertext[i + 1] = matrix[(r2 + 1) % 5][c2];
    }
    else
    {
        ciphertext[i] = matrix[r1][c2];
        ciphertext[i + 1] = matrix[r2][c1];
    }
}

ciphertext[j] = '\0';

printf("\nPrepared Plaintext : %s", prepared);
printf("\nCiphertext      : %s\n", ciphertext);

return 0;
}

```

OUTPUT :

```
1 #include <iostream>
2 #include <string>
3 #include <string.h>
4
5 char keyTable[25] = {
6     'W','Q','M','A','R',
7     'C','N','Y','P','S',
8     'E','H','O','I','K',
9     'L','U','G','V','T',
10    'F','J','X','B','Z'
11 };
12
13 void findPosition(char ch, int *row, int *col)
14 {
15     if(ch == 'J')
16         ch = 'I';
17     for(int i = 0; i < 25; i++)
18     {
19         for(int j = 0; j < 5; j++)
20         {
21             if(keyTable[i*5+j] == ch)
22             {
23                 *row = i;
24                 *col = j;
25                 return;
26             }
27         }
28     }
29 }
30
31 int main()
32 {
33     char plaintext[100];
34     char prepared[100];
35     char ciphertext[100];
36     int i, j = 0;
37     printf("Enter Plaintext: ");
38     scanf("%s", plaintext);
39
40     // Convert to uppercase
41     for(i = 0; plaintext[i] != '\0'; i++)
42         plaintext[i] = toupper(plaintext[i]);
43
44     // Remove plaintext
45     for(i = 0; plaintext[i] != '\0'; i++)
46     {
47         prepared[j++] = plaintext[i];
48         if(plaintext[i] == 'J' || 'I' : plaintext[i]);
49         if(plaintext[i+1] != '\0' & plaintext[i] != 'X')
50             prepared[j++] = (plaintext[i+1] == 'J' || 'I' : toupper(plaintext[i+1]));
51         if(plaintext[i] == 'X')
52             prepared[j++] = 'X';
53     }
54
55     // Encrypt
56     for(i = 0; prepared[i] != '\0'; i++)
57     {
58         int row1, col1, row2, col2;
59         findPosition(prepared[i], &row1, &col1);
60         if(prepared[i+1] != '\0')
61         {
62             findPosition(prepared[i+1], &row2, &col2);
63             // Swap columns
64             int temp = col1;
65             col1 = col2;
66             col2 = temp;
67             ciphertext[i] = keyTable[row1*5+col1];
68             ciphertext[i+1] = keyTable[row2*5+col2];
69         }
70         else
71             ciphertext[i] = keyTable[row1*5+col1];
72     }
73     ciphertext[i] = '\0';
74     printf("Prepared Plaintext : %s\n", prepared);
75     printf("Ciphertext : %s\n", ciphertext);
76     return 0;
77 }
```

Enter Plaintext: HELLO
Prepared Plaintext : HELXLO
Ciphertext : CFSUPM

Process exited after 2.755 seconds with return value 0
Press any key to continue . . .

Compiler | Resources | Compile Log | Debug | Find Results | Console | Close

Abort Compilation

Shorten compiler path

Errors: 0
Warnings: 0
Output Filename: D:\CSA5112 - Cryptography\LAB PROGRAMS\Playfair Cipher.cpp
Output Size: 324.0322265625 KiB
Compilation Time: 0.49s

Line: 94 Col: 2 Sel: 0 Lines: 94 Length: 2102 Insert Done parsing in 0.188 seconds